2026

Tugas 1

OPTIMASI DAN TEKNOLOGI BASIS DATA

CHRISTIAN CHANDRA

Daftar Isi

Optimasi Basis Data Dengan Menerapkan Indexing Untuk Mengambil Data Mencari Total Biaya Dan Tanggal Terakhir Tiap Server Di Lokasi Tertentu 2

Informasi Lingkungan 2

Query baseline 2

Evaluasi pengaruh indeks terhadap mekanisme Join 5

Tabel Perbandingan Performance 5

Analisis & Pembahasan Overhead DML 6

INSERT 6

UPDATE 7

DELETE 9

Nama : Christian Chandra

NPM : 06.2024.1.07763

Link Github : https://github.com/christian1403/optek-basdat-inventarisku

Optimasi Basis Data Dengan Menerapkan Indexing Untuk Mengambil Data Mencari Total Biaya Dan Tanggal Terakhir Tiap Server Di Lokasi Tertentu

Informasi Lingkungan

- RDBMS : PostgreSQL
- Versi PostgreSQL : 18.6
- Dataset : Sistem Inventaris Aset

Query baseline

Query : get_total_biaya_dan_tanggal_terakhir_server

Query untuk mencari total biaya dan tanggal terakhir tiap server di Lokasi id 4 (Data Center Surabaya (DRC))

Query :

```
SELECT a.kode_asset, a.nama_asset, s.nama AS status,
(SELECT SUM(h.biaya) FROM asset_history h WHERE h.id_asset = a.id) AS
total_biaya,
 (SELECT MAX(h.tanggal) FROM asset_history h WHERE h.id_asset = a.id) AS
history_terakhir
FROM asset a
JOIN status s ON s.id = a.id_status
JOIN lokasi l ON l.id = a.id_lokasi
WHERE a.id_kategori = 5
  AND l.id = 4
  AND a.tanggal_pembelian BETWEEN '2024-01-01' AND '2024-12-31'
ORDER BY total_biaya DESC NULLS LAST;
```

Hasil EXPLAIN (ANALYZE, BUFFERS) :

```
QUERY PLAN
-----------------------
```

2. Filter utama

```
->  Parallel Seq Scan on asset a  (cost=0.00..16614.17 rows=1 width=56) (actual time=21.828..27.118 rows=41.00 loops=3)
      Filter: ((tanggal_pembelian >= '2024-01-01'::date) AND (tanggal_pembelian <= '2024-12-31'::date) AND (id_lokasi = 4) AND (id_kategori = 5))
      Rows Removed by Filter: 83246
      Buffers: shared hit=14140 read=392
->  Seq Scan on status s  (cost=0.00..1.07 rows=7 width=13) (actual time=0.001..0.001 rows=2.06 loops=123)
      Buffers: shared hit=123
```

Bisa terlihat bahwa disini ada Seq Scan untuk table asset dengan actual time 21.828..27.118 dan loops 3x

Berarti kita ambil tengah2 nya mungkin, 27 *3 = sudah 81 ms

Mungkin ini tidak sebanyak yang pertama, tapi kita juga bisa mengoptimalkan untuk ini yaitu dengan composite index

```
CREATE INDEX idx_asset_lokasi_kategori_tanggal ON asset (id_lokasi, id_kategori, tanggal_pembelian);
```

*Kenapa tidak menggunakan partial / filtered index, karena disini value where nya dinamis dan tidak statis, sehingga tidak memungkinkan menggunakan partial / filtered index.

```
| Planning:
|   Buffers: shared hit=4
| Planning Time: 0.361 ms
| JIT:
|   Functions: 44
|   Options: Inlining false, Optimization false, Expressions true, Deforming true
|   Timing: Generation 2.484 ms (Deform 1.412 ms), Inlining 0.000 ms, Optimization 1.468 ms, Emission 22.413 ms, Total 26.364 ms
| Execution Time: 13520.467 ms
```

Bisa terlihat disini bahwa execution time nya sekitar 13 detik, sesuai dengan analisa di atas.

Hasil EXPLAIN (ANALYZE, BUFFERS) Setelah index dipasang :

```
| QUERY PLAN
| ------------------
```

Disini bisa terlihat bahwa 3 proses tadi yang actual time nya besar sekarang sudah jauh lebih kecil

1. 2 sub query di table asset_history

```
SubPlan 1
  ->  Aggregate  (cost=23.34..23.35 rows=1 width=32) (actual time=0.074..0.074 rows=1.00 loops=123)
        Buffers: shared hit=337 read=461
        ->  Index Scan using idx_asset_history_id_asset on asset_history h  (cost=0.43..23.32 rows=6 width=4) (actual time=0.030..0.072 rows=3.49 loops=123)
              Index Cond: (id_asset = a.id)
              Index Searches: 123
              Buffers: shared hit=337 read=461
SubPlan 2
  ->  Aggregate  (cost=23.34..23.35 rows=1 width=4) (actual time=0.003..0.003 rows=1.00 loops=123)
        Buffers: shared hit=798
        ->  Index Scan using idx_asset_history_id_asset on asset_history h_1  (cost=0.43..23.32 rows=6 width=4) (actual time=0.001..0.002 rows=3.49 loops=123)
              Index Cond: (id_asset = a.id)
              Index Searches: 123
              Buffers: shared hit=798
```

Disini sudah terlihat bahwa dari Seq scan berubah menjadi Index scan dan actual time nya jauh lebih kecil

2. Filter utama

```
->  Materialize  (cost=0.42..12.48 rows=2 width=56) (actual time=0.011..0.080 rows=123.00 loops=7)
      Storage: Memory  Maximum Storage: 27kB
      Buffers: shared hit=123 read=4
      ->  Index Scan using idx_asset_lokasi_kategori_tanggal on asset a  (cost=0.42..12.47 rows=2 width=56) (actual time=0.076..0.496 rows=123.00 loops=1)
            Index Cond: ((id_lokasi = 4) AND (id_kategori = 5) AND (tanggal_pembelian >= '2024-01-01'::date) AND (tanggal_pembelian <= '2024-12-31'::date))
            Index Searches: 1
            Buffers: shared hit=123 read=4
```

Disini juga ada perubahan yang awalnya dari Seq scan menjadi Index scan, dan actual time nya jauh lebih kecil.

```
Planning:
  Buffers: shared hit=357 read=3
Planning Time: 1.651 ms
Execution Time: 10.373 ms
```

Execution time nya juga berkurang drastis menjadi 0.010 detik.

Evaluasi pengaruh indeks terhadap mekanisme Join

Algoritma join di kedua plan sama-sama Nested Loop. Indeks tidak mengubah jenisnya, tetapi mengubah sisi inner dari full scan berulang menjadi lookup B-tree (Index Nested Loop). Itulah sumber percepatan +-1.300×. Dan juga Indeks komposit asset mengubah access path dan urutan join (serta menghilangkan parallel scan), tetapi kontribusinya ke waktu total kecil (±27 ms). Indeks asset_history(id_asset) adalah indeks yang paling menentukan, karena menghapus 246 full table scan

Tabel Perbandingan Performance

Query get_total_biaya_dan_tanggal_terakhir_server			
Execution Time sebelum indexing		13520.467 ms (+- 13 detik)	
Estimated Total Cost sebelum indexing		188105.14	
Strategi Index			
Index Tunggal		Index Composite	
Nama Index	***idx_asset_history_id_asset***	Nama Index	***idx_asset_lokasi_kategori_tanggal***
Table	***asset_history***	Table	***asset***
Column	***id_asset***	Columns	***id_lokasi, id_kategori, tanggal_pembelian***

Size	15 MB	Size	6808 kB
Execution Time setelah indexing		10.373 ms (+- 0.010 detik)	
Estimated Total Cost setelah indexing		108.81	

Analisis & Pembahasan Overhead DML

Disini kita ambil ke index yang paling berpengaruh yaitu ***idx_asset_history_id_asset*** (Index Tunggal B-Tree)

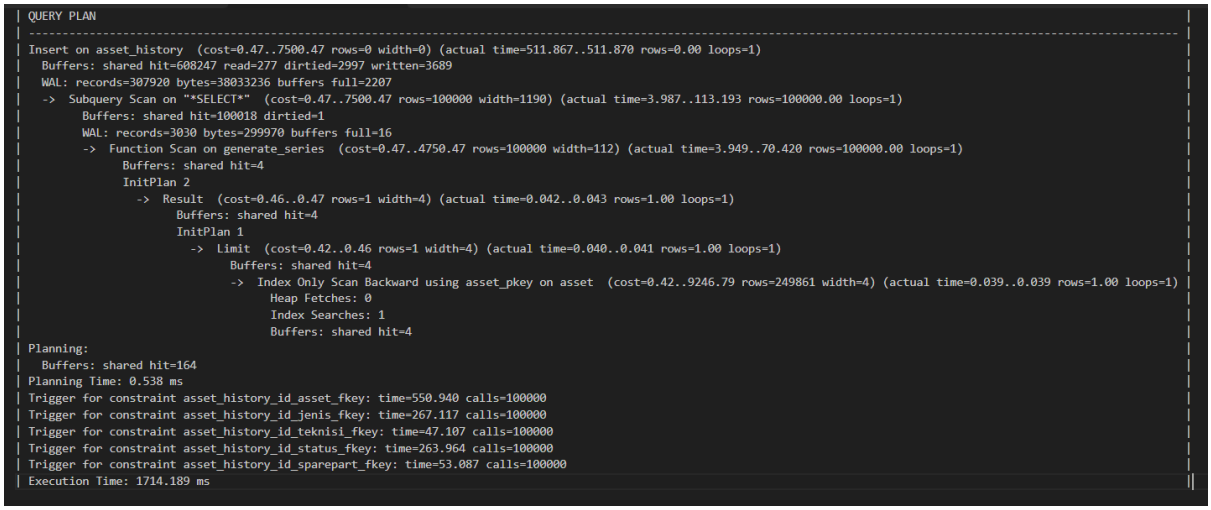
INSERT

```
BEGIN;
-- DROP INDEX idx_asset_history_id_asset;
EXPLAIN (ANALYZE, BUFFERS, WAL)
INSERT INTO asset_history (id_asset, tanggal, id_jenis, id_status,
keluhan, tindakan, biaya)
SELECT 1 + floor(random() * (SELECT max(id) FROM asset))::int,
       current_date - (random() * 700)::int,
       1, 1,
       repeat('keluhan uji ', 5),
       repeat('tindakan uji ', 5),
       round((random() * 5000000)::numeric, 2)
FROM generate_series(1, 100000);
ROLLBACK;

BEGIN;
DROP INDEX idx_asset_history_id_asset;
EXPLAIN (ANALYZE, BUFFERS, WAL)
INSERT INTO asset_history (id_asset, tanggal, id_jenis, id_status,
keluhan, tindakan, biaya)
SELECT 1 + floor(random() * (SELECT max(id) FROM asset))::int,
       current_date - (random() * 700)::int,
       1, 1,
       repeat('keluhan uji ', 5),
       repeat('tindakan uji ', 5),
       round((random() * 5000000)::numeric, 2)
FROM generate_series(1, 100000);
ROLLBACK;
```

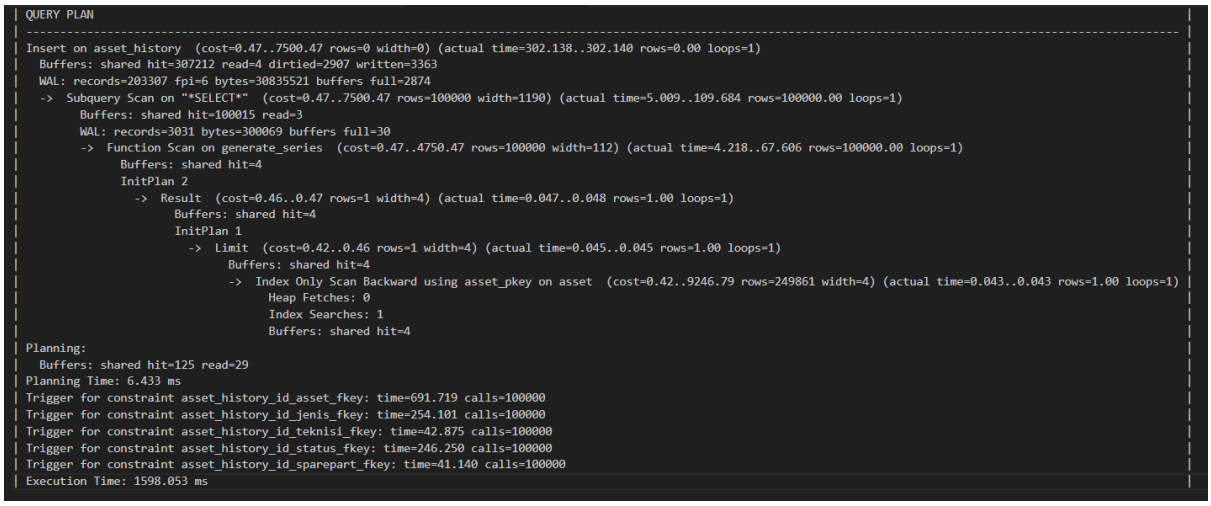
Hasil :

Tanpa Drop Index



```
QUERY PLAN
Insert on asset_history  (cost=0.47..7500.47 rows=0 width=0) (actual time=511.867..511.870 rows=0.00 loops=1)
  Buffers: shared hit=608247 read=277 dirtied=2997 written=3689
  WAL: records=307920 bytes=38033236 buffers full=2207
  ->  Subquery Scan on "*SELECT*"  (cost=0.47..7500.47 rows=100000 width=1190) (actual time=3.987..113.193 rows=100000.00 loops=1)
        Buffers: shared hit=100018 dirtied=1
        WAL: records=3030 bytes=299970 buffers full=16
        ->  Function Scan on generate_series  (cost=0.47..4750.47 rows=100000 width=112) (actual time=3.949..70.420 rows=100000.00 loops=1)
              Buffers: shared hit=4
              InitPlan 2
                ->  Result  (cost=0.46..0.47 rows=1 width=4) (actual time=0.042..0.043 rows=1.00 loops=1)
                      Buffers: shared hit=4
                      InitPlan 1
                        ->  Limit  (cost=0.42..0.46 rows=1 width=4) (actual time=0.040..0.041 rows=1.00 loops=1)
                              Buffers: shared hit=4
                              ->  Index Only Scan Backward using asset_pkey on asset  (cost=0.42..9246.79 rows=249861 width=4) (actual time=0.039..0.039 rows=1.00 loops=1)
                                    Heap Fetches: 0
                                    Index Searches: 1
                                    Buffers: shared hit=4
Planning:
  Buffers: shared hit=164
Planning Time: 0.538 ms
Trigger for constraint asset_history_id_asset_fkey: time=550.940 calls=100000
Trigger for constraint asset_history_id_jenis_fkey: time=267.117 calls=100000
Trigger for constraint asset_history_id_teknisi_fkey: time=47.107 calls=100000
Trigger for constraint asset_history_id_status_fkey: time=263.964 calls=100000
Trigger for constraint asset_history_id_sparepart_fkey: time=53.087 calls=100000
Execution Time: 1714.189 ms
```

Dengan Drop Index



```
QUERY PLAN
Insert on asset_history  (cost=0.47..7500.47 rows=0 width=0) (actual time=302.138..302.140 rows=0.00 loops=1)
  Buffers: shared hit=307212 read=4 dirtied=2907 written=3363
  WAL: records=203307 fpi=6 bytes=30835521 buffers full=2874
  ->  Subquery Scan on "*SELECT*"  (cost=0.47..7500.47 rows=100000 width=1190) (actual time=5.009..109.684 rows=100000.00 loops=1)
        Buffers: shared hit=100015 read=3
        WAL: records=3031 bytes=300069 buffers full=30
        ->  Function Scan on generate_series  (cost=0.47..4750.47 rows=100000 width=112) (actual time=4.218..67.606 rows=100000.00 loops=1)
              Buffers: shared hit=4
              InitPlan 2
                ->  Result  (cost=0.46..0.47 rows=1 width=4) (actual time=0.047..0.048 rows=1.00 loops=1)
                      Buffers: shared hit=4
                      InitPlan 1
                        ->  Limit  (cost=0.42..0.46 rows=1 width=4) (actual time=0.045..0.045 rows=1.00 loops=1)
                              Buffers: shared hit=4
                              ->  Index Only Scan Backward using asset_pkey on asset  (cost=0.42..9246.79 rows=249861 width=4) (actual time=0.043..0.043 rows=1.00 loops=1)
                                    Heap Fetches: 0
                                    Index Searches: 1
                                    Buffers: shared hit=4
Planning:
  Buffers: shared hit=125 read=29
Planning Time: 6.433 ms
Trigger for constraint asset_history_id_asset_fkey: time=691.719 calls=100000
Trigger for constraint asset_history_id_jenis_fkey: time=254.101 calls=100000
Trigger for constraint asset_history_id_teknisi_fkey: time=42.875 calls=100000
Trigger for constraint asset_history_id_status_fkey: time=246.250 calls=100000
Trigger for constraint asset_history_id_sparepart_fkey: time=41.140 calls=100000
Execution Time: 1598.053 ms
```

Execution Time		
Drop Index	Tanpa Drop Index	Selisih
1598.053 ms	1714.189 ms	116.136 ms (7.27%)

Disini menunjukkan bahwa perintah DML Insert ke table asset_history yang memiliki index B-Tree dengan mensimulasikan insert 100rb data, waktu execution time nya lebih lambat sekitar 116.136 ms atau mengalami kenaikan execution time sekitar 7.27% daripada tidak memiliki index.

UPDATE

```
BEGIN;
-- DROP INDEX idx_asset_history_id_asset;
```

```
EXPLAIN (ANALYZE, BUFFERS, WAL)
UPDATE asset_history
SET biaya = coalesce(biaya, 0) + 50000,
    update_at = now()
WHERE id_asset BETWEEN 1 AND 100000;

ROLLBACK;

BEGIN;
DROP INDEX idx_asset_history_id_asset;

EXPLAIN (ANALYZE, BUFFERS, WAL)
UPDATE asset_history
SET biaya = coalesce(biaya, 0) + 50000,
    update_at = now()
WHERE id_asset BETWEEN 1 AND 100000;

ROLLBACK;
```

Hasil :

Tanpa drop index

```
QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------
Update on asset_history  (cost=0.00..177871.30 rows=0 width=0) (actual time=10663.998..10663.999 rows=0.00 loops=1)
  Buffers: shared hit=8981557 read=115811 dirtied=59863 written=62021
  WAL: records=3075898 fpi=41660 bytes=631650846 buffers full=56511
  ->  Seq Scan on asset_history  (cost=0.00..177871.30 rows=2502094 width=32) (actual time=2.361..3166.667 rows=755064.00 loops=1)
        Filter: ((id_asset >= 1) AND (id_asset <= 100000))
        Rows Removed by Filter: 559013
        Buffers: shared hit=4884 read=82337 dirtied=16245 written=14367
        WAL: records=17531 fpi=16245 bytes=133152173 buffers full=11844
Planning:
  Buffers: shared hit=3 read=2 dirtied=1
Planning Time: 0.128 ms
JIT:
  Functions: 5
  Options: Inlining false, Optimization false, Expressions true, Deforming true
  Timing: Generation 0.281 ms (Deform 0.080 ms), Inlining 0.000 ms, Optimization 0.209 ms, Emission 2.684 ms, Total 3.174 ms
Execution Time: 10664.335 ms
```

Dengan drop index

```
QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------------------------
Update on asset_history  (cost=0.00..208977.98 rows=0 width=0) (actual time=2960.413..2960.415 rows=0.00 loops=1)
  Buffers: shared hit=6082577 read=108555 dirtied=38287 written=47583
  WAL: records=2263194 fpi=16972 bytes=386303144 buffers full=42719
  ->  Seq Scan on asset_history  (cost=0.00..208977.98 rows=2939226 width=32) (actual time=3.342..468.968 rows=755064.00 loops=1)
        Filter: ((id_asset >= 1) AND (id_asset <= 100000))
        Rows Removed by Filter: 559013
        Buffers: shared hit=8 read=102469 written=17115
Planning:
  Buffers: shared hit=52 read=11
Planning Time: 0.240 ms
JIT:
  Functions: 5
  Options: Inlining false, Optimization false, Expressions true, Deforming true
  Timing: Generation 0.350 ms (Deform 0.132 ms), Inlining 0.000 ms, Optimization 0.224 ms, Emission 3.521 ms, Total 4.095 ms
Execution Time: 2972.731 ms
```

Execution Time		
Drop Index	Tanpa Drop Index	Selisih
2972.731 ms	10664.335 ms	7691.604 ms (258.73%)

Disini menunjukkan bahwa perintah DML Update ke table asset_history yang memiliki index B-Tree dengan mensimulasikan update 100rb data, waktu execution time nya lebih lambat sekitar 7691.604 ms atau mengalami kenaikan execution time sekitar 258.73% daripada tidak memiliki index.

DELETE

```
BEGIN;
-- DROP INDEX idx_asset_history_id_asset;

EXPLAIN (ANALYZE, BUFFERS, WAL)
DELETE FROM asset_history
WHERE id_asset BETWEEN 1 AND 100000;

ROLLBACK;

BEGIN;
DROP INDEX idx_asset_history_id_asset;

EXPLAIN (ANALYZE, BUFFERS, WAL)
DELETE FROM asset_history
WHERE id_asset BETWEEN 1 AND 100000;

ROLLBACK;
```

Hasil :

Tanpa drop index

```
| QUERY PLAN                                                                                                                                                 |
| ---------------------------------------------------------------------------------------------------------------------------------------------------------- |
| Delete on asset_history  (cost=0.00..55558.46 rows=0 width=0) (actual time=420.786..420.787 rows=0.00 loops=1)                                             |
|   Buffers: shared hit=755065 read=26195 dirtied=16971 written=11450                                                                                        |
|   WAL: records=755064 bytes=40773456 buffers full=4331                                                                                                     |
|   ->  Seq Scan on asset_history  (cost=0.00..55558.46 rows=1119432 width=6) (actual time=0.043..226.990 rows=755064.00 loops=1)                            |
|         Filter: ((id_asset >= 1) AND (id_asset <= 100000))                                                                                                 |
|         Rows Removed by Filter: 559013                                                                                                                     |
|         Buffers: shared hit=1 read=26195 dirtied=16971 written=11450                                                                                       |
| Planning:                                                                                                                                                  |
|   Buffers: shared hit=61 read=25 dirtied=1                                                                                                                 |
| Planning Time: 6.008 ms                                                                                                                                    |
| Execution Time: 421.387 ms                                                                                                                                 |
```

Dengan drop index

```
| QUERY PLAN                                                                                                                                                 |
| ---------------------------------------------------------------------------------------------------------------------------------------------------------- |
| Delete on asset_history  (cost=0.00..55558.46 rows=0 width=0) (actual time=407.989..407.991 rows=0.00 loops=1)                                             |
|   Buffers: shared hit=755064 read=26196 dirtied=16972 written=16007                                                                                        |
|   WAL: records=755064 bytes=40773456 buffers full=3915                                                                                                     |
|   ->  Seq Scan on asset_history  (cost=0.00..55558.46 rows=1119236 width=6) (actual time=0.156..218.019 rows=755064.00 loops=1)                            |
|         Filter: ((id_asset >= 1) AND (id_asset <= 100000))                                                                                                 |
|         Rows Removed by Filter: 559013                                                                                                                     |
|         Buffers: shared read=26196 dirtied=16972 written=16007                                                                                             |
| Planning:                                                                                                                                                  |
|   Buffers: shared hit=3 read=1                                                                                                                             |
| Planning Time: 0.065 ms                                                                                                                                    |
| Execution Time: 408.011 ms                                                                                                                                 |
```

Execution Time		
Drop Index	Tanpa Drop Index	Selisih
408.011 ms	421.387 ms	13.376 ms (3.28%)

Disini menunjukkan bahwa perintah DML Delete ke table asset_history yang memiliki index B-Tree dengan mensimulasikan delete 100rb data, waktu execution time nya lebih lambat sekitar 13.376 ms atau mengalami kenaikan execution time sekitar 3.28% daripada tidak memiliki index.